

Here is a complete, step-by-step guide to capturing HTTP, FTP, and DNS traffic, isolating unencrypted FTP credentials, and analyzing an `hping3` SYN flood attack using Wireshark.

1. Capturing HTTP, FTP, and DNS Traffic

To capture traffic for these specific protocols in Wireshark, you can use **Display Filters** after starting a live capture or apply a **Capture Filter** beforehand.

Option A: Using Display Filters (Post-Capture)

Start a standard capture on your active interface (e.g., `eth0` or `wlan0`), generate traffic, and apply one of the following filters in the top Wireshark search bar:

- Capture HTTP only: `http`
- Capture FTP only: `ftp`
- Capture DNS only: `dns`
- Combine all three:

Plaintext



```
http || ftp || dns
```

Option B: Using Capture Filters (Pre-Capture)

If you want Wireshark to *only* record packets for these protocols to save memory, set this in **Capture Options** (`Ctrl + K`):

Plaintext



```
port 80 or port 21 or port 53
```

2. Filtering Credentials from Unencrypted FTP Traffic

FTP transmits authentication details in plain text across the network using the commands `USER` and `PASS`.

Step-by-Step Credential Extraction

1. Filter FTP Authentication Packets:

In Wireshark's display filter, enter:

Plaintext



```
ftp.request.command == "USER" || ftp.request.command == "PASS"
```

1. `USER` packet: Contains the plaintext username.
2. `PASS` packet: Contains the plaintext password.

2. Inspect the TCP Stream:

Right-click any FTP packet → **Follow** → **TCP Stream**.

A pop-up window will show the entire conversation in cleartext:

Plaintext



```
220 Welcome to FTP service.  
USER admin  
331 Please specify the password.  
PASS SecretPassword123  
230 Login successful.
```

3. Analyzing a SYN Flood Attack in Wireshark

When simulating a SYN flood using `hping3` from Kali Linux:

Bash



```
sudo hping3 -S -p 80 --flood --rand-source <TARGET_IP>
```

Key Indicators to Look For in Wireshark

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.0.2	10.128.0.2	TCP	54	3341 → 80 [SYN] Seq=0 Win=0 Len=0
2	0.000007	10.128.0.2	10.0.0.2	TCP	58	80 → 3222 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460
3	0.000014	10.128.0.2	10.0.0.2	TCP	58	80 → 3341 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460
4	0.000029	10.0.0.2	10.128.0.2	TCP	54	3342 → 80 [SYN] Seq=0 Win=0 Len=0
5	0.010233	10.128.0.2	10.0.0.2	TCP	58	80 → 3220 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460
6	0.014072	10.128.0.2	10.0.0.2	TCP	58	80 → 3342 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460
7	0.018030	10.0.0.2	10.128.0.2	TCP	54	3343 → 80 [SYN] Seq=0 Win=0 Len=0
8	0.022220	10.128.0.2	10.0.0.2	TCP	58	80 → 3343 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460
9	0.023466	10.128.0.2	10.0.0.2	TCP	58	80 → 3210 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460
10	0.025243	10.0.0.2	10.128.0.2	TCP	54	3344 → 80 [SYN] Seq=0 Win=0 Len=0
11	0.026672	10.128.0.2	10.0.0.2	TCP	58	80 → 3210 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460
12	0.028030	10.128.0.2	10.0.0.2	TCP	58	80 → 3221 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460
13	0.030523	10.128.0.2	10.0.0.2	TCP	58	80 → 3344 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460

Frame 1: 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface 0
Ethernet II, Src: 42:01:0a:f0:00:17 (42:01:0a:f0:00:17), Dst: 42:01:0a:f0:00:01 (42:01:0a:f0:00:01)
Internet Protocol Version 4, Src: 10.0.0.2, Dst: 10.128.0.2
Transmission Control Protocol, Src Port: 3341, Dst Port: 80, Seq: 0, Len: 0
Source Port: 3341
Destination Port: 80
[Stream index: 0]
[TCP Segment Len: 0]
Sequence number: 0 (relative sequence number)
[Next sequence number: 0 (relative sequence number)]
Acknowledgment number: 102350004
[Window size: 0] (relative window size)
[Flags: 0x002 (SYN)]
Window size value: 0
[Calculated window size: 0]
Checksum: 0x0000 (unverified)
[Checksum status: Unverified]
Urgent pointer: 0
[Timestamps]

High-volume TCP SYN flood capture in Wireshark.
Source: MazeBolt

Wireshark Filters for SYN Flood Analysis

1. Filter Incoming SYN Requests (Without Acknowledgments):

Plaintext



```
tcp.flags.syn == 1 && tcp.flags.ack == 0
```

1. *Observation:* You will see hundreds or thousands of packets per second targeting port 80/443.

2. Filter Target SYN-ACK Responses:

Plaintext



```
tcp.flags.syn == 1 && tcp.flags.ack == 1
```

1. *Observation:* The target server sends **SYN-ACK** responses, but because the attacker used `--rand-source`, these go to non-existent or unresponsive hosts.

3. Check for Handshake Completion (ACKs):

Plaintext



```
tcp.flags.syn == 0 && tcp.flags.ack == 1 && tcp.dstport == 80
```

1. *Observation:* Under a SYN flood, **almost zero ACK packets return**, proving that the three-way handshake is never completed.

Wireshark Graphical Analysis Tools

- **I/O Graph** (**Statistics** → **I/O Graph**): Add a filter for `tcp.flags.syn == 1`. You will see a massive vertical spike in packet velocity (packets per second).
- **Conversations** (**Statistics** → **Conversations** → **TCP**): You will see an abnormally long list of single-packet TCP sessions from varying source IP addresses targeting a single destination IP.

Where would you like to go next with network packet analysis?

Filter HTTP POST requests to find submitted web credentials



Export Wireshark summary statistics to CSV

